

JouleOps @NorthWind Manufacturing

Agentic AI Enterprise Assistant

Output Screenshots

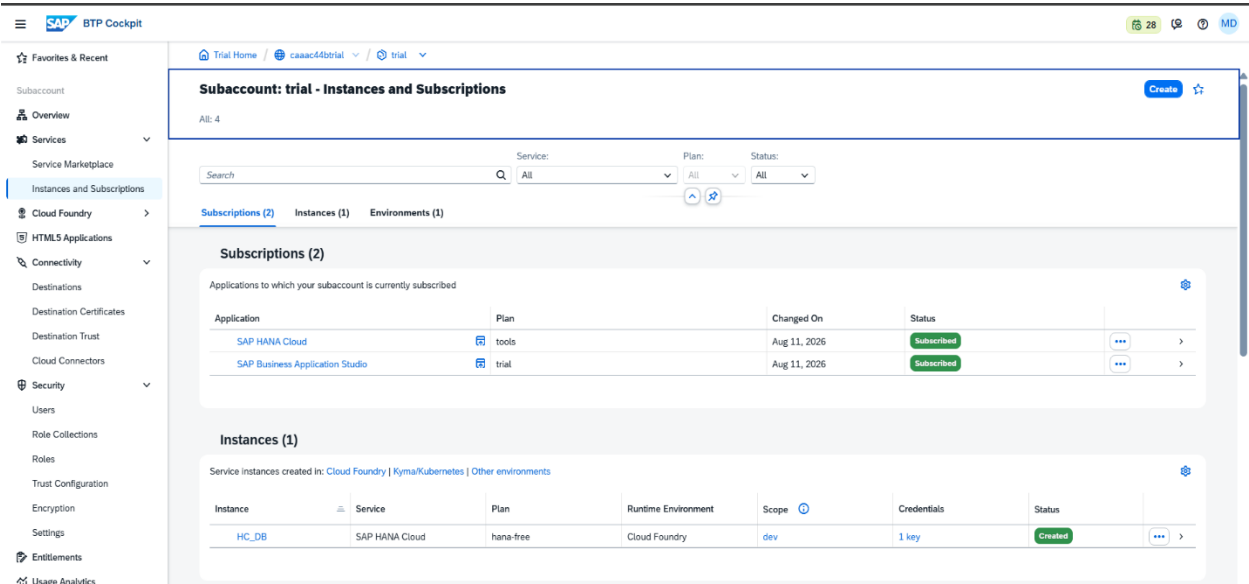


Fig1: Instance creation/SAP Hana Cloud subscription in cockpit

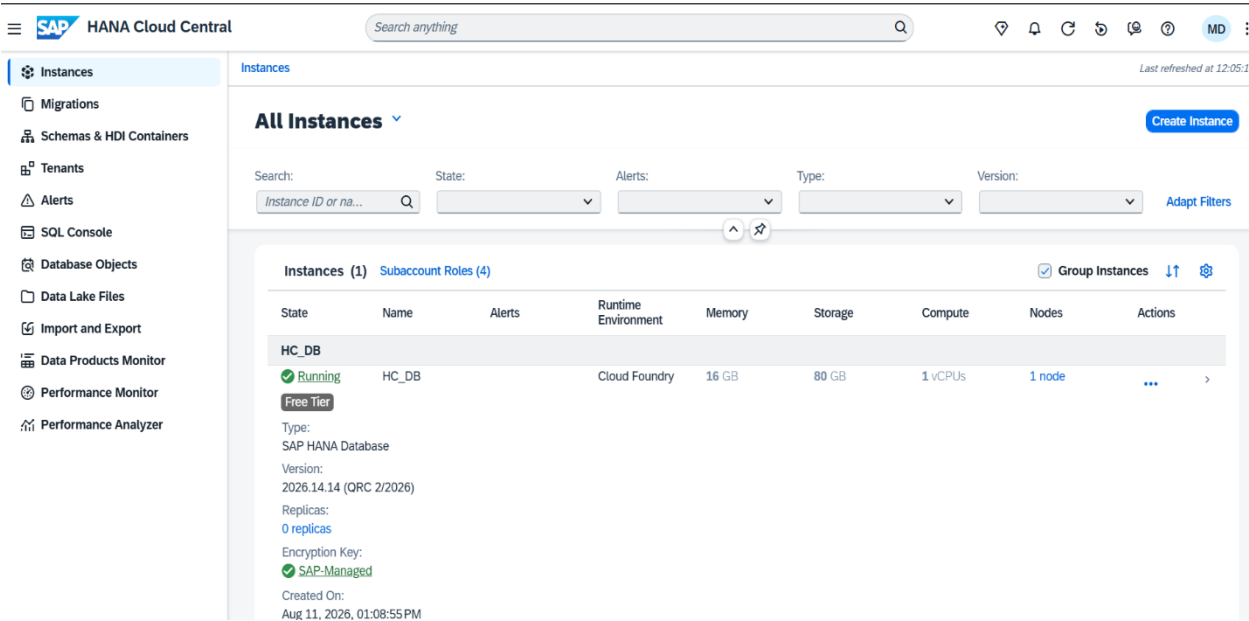


Fig2: Instance running

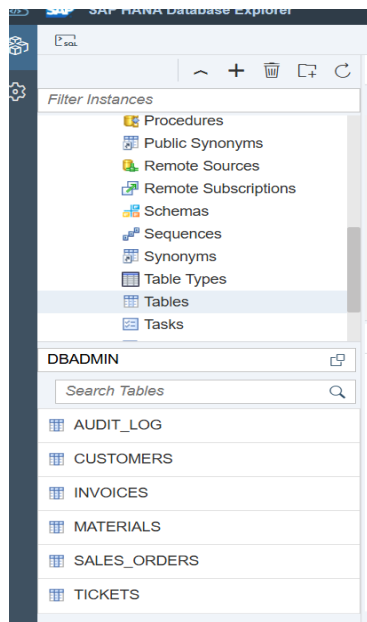


Fig 3: Tables Created in SAP Hana Cloud

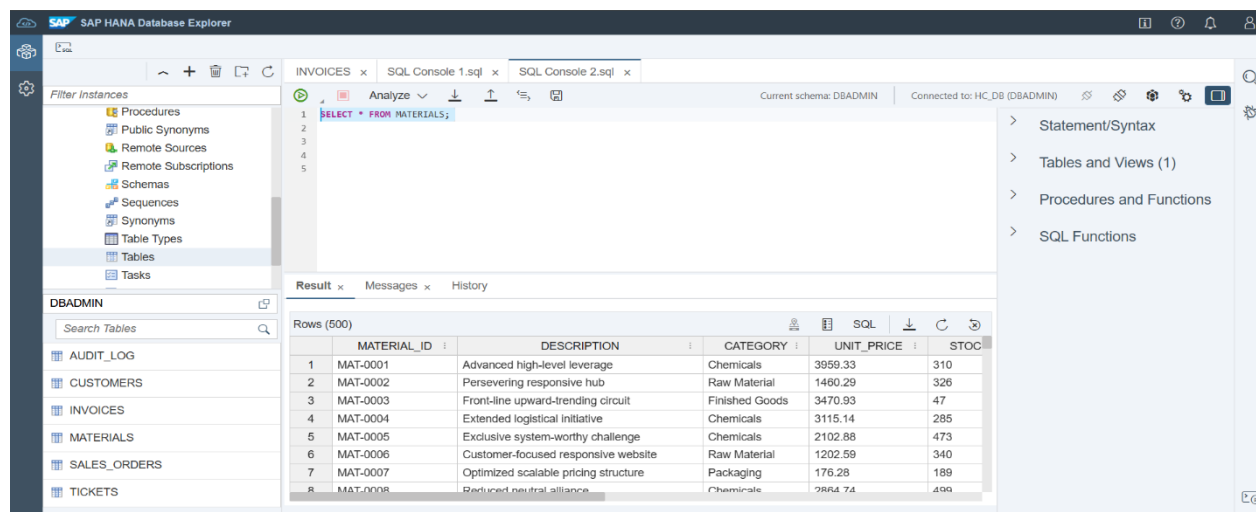
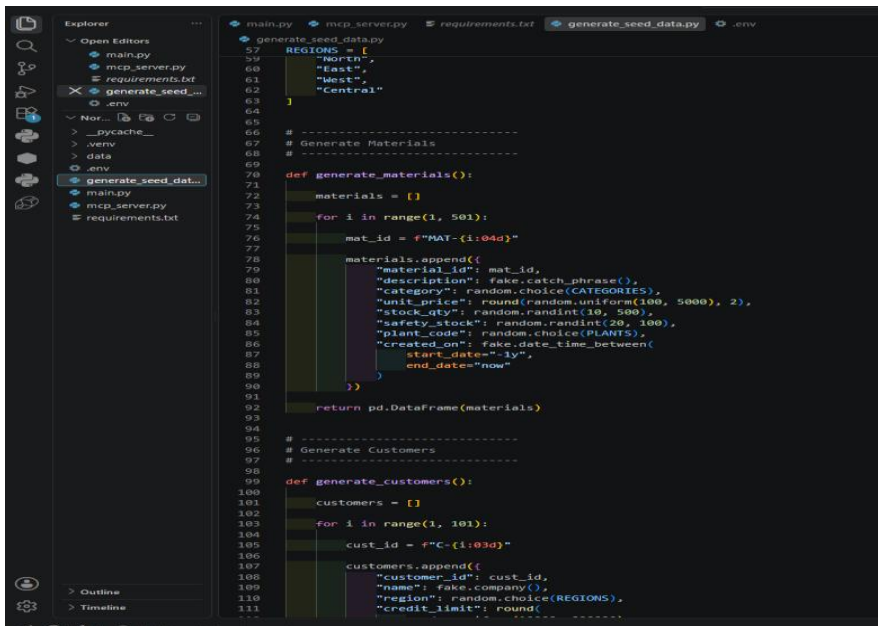


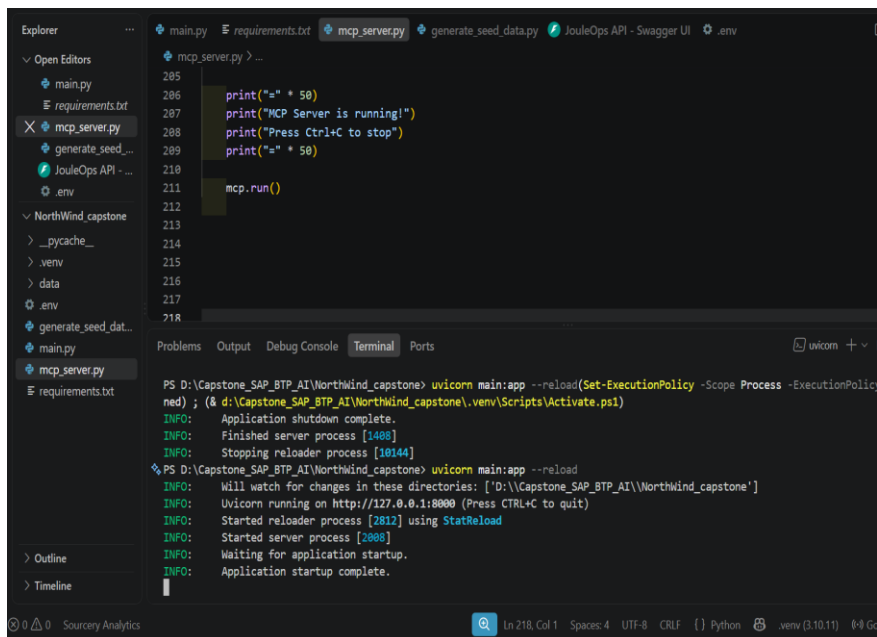
Fig4: Inserted Records in Material Table



The screenshot shows the Visual Studio Code editor with the file explorer on the left and the code editor on the right. The file explorer shows a project structure with files like main.py, mcp_server.py, requirements.txt, generate_seed_data.py, .env, data, and .venv. The code editor displays the generate_seed_data.py script, which defines functions to generate materials and customers. The script includes comments and uses random functions to generate data.

```
57 REGIONS = [
58     "North",
59     "East",
60     "West",
61     "Central"
62 ]
63
64 # -----
65 # Generate Materials
66 # -----
67
68 def generate_materials():
69     materials = []
70     for i in range(1, 501):
71         mat_id = f"MAT-{i:04d}"
72         materials.append({
73             "material_id": mat_id,
74             "description": fake.catch_phrase(),
75             "category": random.choice(CATEGORIES),
76             "unit_price": round(random.uniform(100, 5000), 2),
77             "stock_qty": random.randint(10, 500),
78             "safety_stock": random.randint(20, 100),
79             "plant_code": random.choice(PLANTS),
80             "created_on": fake.date_time_between(
81                 start_date="1y",
82                 end_date="now"
83             )
84         })
85     return pd.DataFrame(materials)
86
87 # -----
88 # Generate Customers
89 # -----
90
91 def generate_customers():
92     customers = []
93     for i in range(1, 101):
94         cust_id = f"C-{i:03d}"
95         customers.append({
96             "customer_id": cust_id,
97             "name": fake.company(),
98             "region": random.choice(REGIONS),
99             "credit_limit": round(
100                 random.uniform(1000, 10000), 2
101             )
102         })
103     return pd.DataFrame(customers)
```

Fig 5: Seed Data Generation in vs code and stored as csv file in data folder



The screenshot shows the Visual Studio Code editor with the file explorer on the left and the code editor on the right. The file explorer shows a project structure with files like main.py, requirements.txt, mcp_server.py, generate_seed_data.py, .env, data, and .venv. The code editor displays the mcp_server.py script, which includes a print statement and a call to mcp.run(). The bottom panel shows the output of the script, indicating that the MCP Server is running and listening on port 8000.

```
285
286 print("s" * 50)
287 print("MCP Server is running!")
288 print("Press Ctrl+C to stop")
289 print("s" * 50)
290
291 mcp.run()
```

PS D:\Capstone_SAP_BTP_AI\NorthWind_capstone> uvicorn main:app --reload (Set-ExecutionPolicy -Scope Process -ExecutionPolicy RemoteSigned) ; (& d:\Capstone_SAP_BTP_AI\NorthWind_capstone\.venv\Scripts\Activate.ps1)

INFO: Application shutdown complete.

INFO: Finished server process [1408]

INFO: Stopping reload process [10144]

PS D:\Capstone_SAP_BTP_AI\NorthWind_capstone> uvicorn main:app --reload

INFO: Will watch for changes in these directories: ['D:\Capstone_SAP_BTP_AI\NorthWind_capstone']

INFO: Uvicorn running on http://127.0.0.1:8000 (Press CTRL+C to quit)

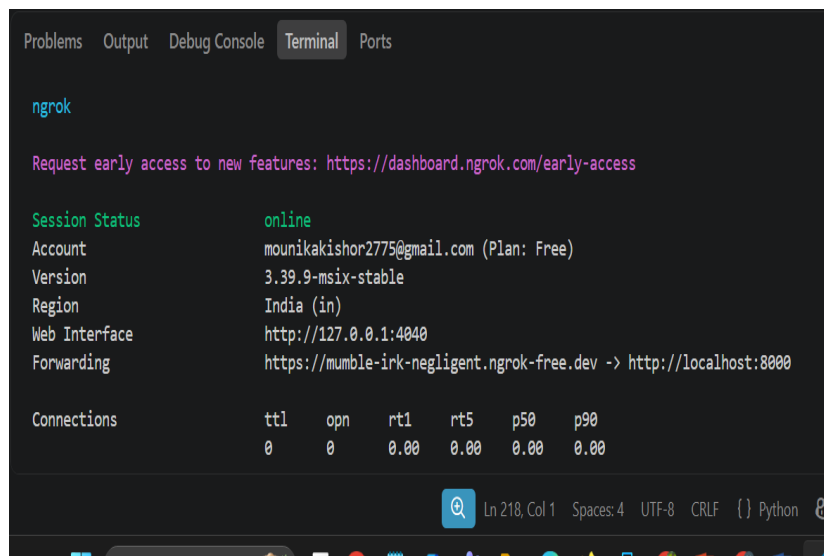
INFO: Started reload process [2812] using StatReload

INFO: Started server process [2008]

INFO: Waiting for application startup.

INFO: Application startup complete.

Fig 6 : Fast API running in vs code



The terminal window displays the ngrok interface. At the top, there are tabs for Problems, Output, Debug Console, Terminal, and Ports. The Terminal tab is active, showing the ngrok logo and a link to request early access to new features: <https://dashboard.ngrok.com/early-access>. Below this, the Session Status is shown as 'online'. The Account is 'mounikakishor2775@gmail.com (Plan: Free)'. The Version is '3.39.9-msix-stable'. The Region is 'India (in)'. The Web Interface is 'http://127.0.0.1:4040'. The Forwarding URL is 'https://mumble-irk-negligent.ngrok-free.dev -> http://localhost:8000'. At the bottom, there is a table of Connections:

Connections	t1	opn	rt1	rt5	p50	p90
	0	0	0.00	0.00	0.00	0.00

The status bar at the bottom indicates 'Ln 218, Col 1', 'Spaces: 4', 'UTF-8', 'CRLF', and 'Python'.

Fig 7: deployed in ngrok

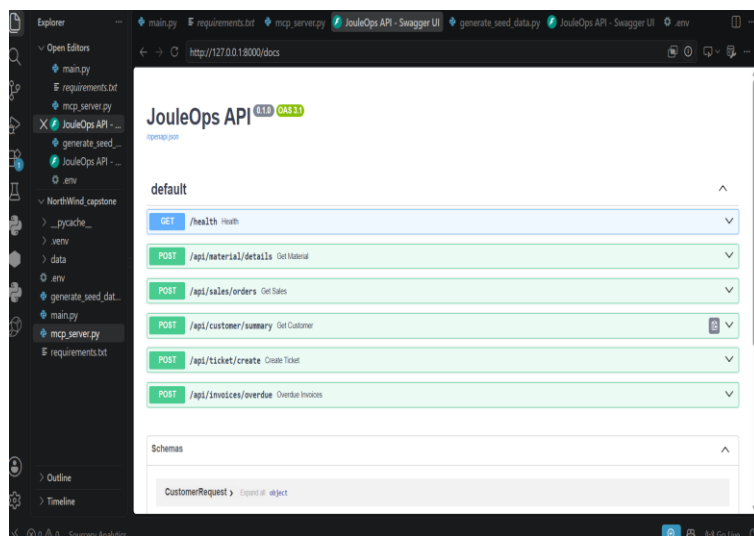


Fig 8: Fast API Endpoints

POST /api/material/details Get Material

Parameters

No parameters

Request body required application/json

Edit Value | Schema

```
{
  "material_id": "MAT-0019",
  "plant_code": "PLT-COI"
}
```

Fig 9: Post Request for getting material data for (Mat-0019) from Hana cloud

```
http://127.0.0.1:8080/api/material/details \
-H 'accept: application/json' \
-H 'Content-Type: application/json' \
-d '{
  "material_id": "MAT-0019",
  "plant_code": "PLT-COI"
}'
```

Request URL

http://127.0.0.1:8080/api/material/details

Server response

Code Details

200

Response body

```
{
  "material_id": "MAT-0019",
  "description": "Organic regional product",
  "category": "Spare Parts",
  "unit_price": 3240.49,
  "stock_qty": 265,
  "safety_stock": 34,
  "plant_code": "PLT-COI",
  "is_below_safety": false,
  "tool": "get_material_details",
  "source_tables": [
    "MATERIALS"
  ]
}
```

Response headers

```
content-length: 254
content-type: application/json
date: Fri,14 Aug 2026 06:54:01 GMT
server: iad.com
```

Fig 10 : Response from SAP Hana Cloud

SAP HANA Database Explorer

Filter Instances

DBADMIN

Search Tables

AUDIT_LOG

CUSTOMERS

INVOICES

MATERIALS

SALES_ORDERS

TICKETS

INVOICES x SQL Console 1.sql x SQL Console 2.sql x

Analyze

1 select * from materials;

Current schema: DBADMIN Connected to: HC_DB (DBADMIN)

Result x Messages x History

Rows (500)

	MATERIAL_ID	DESCRIPTION	CATEGORY	UNIT_PRICE	STOCK
17	MAT-0017	Fully-configurable directional algorithm	Raw Material	1916.00	415
18	MAT-0018	Synchronized holistic concept	Packaging	3915.58	19
19	MAT-0019	Organic regional product	Spare Parts	3240.49	265
20	MAT-0020	Automated well-modulated open architecture	Packaging	2170.68	45
21	MAT-0021	Exclusive bottom-line hardware	Chemicals	3610.64	492
22	MAT-0022	Profound explicit Local Area Network	Spare Parts	612.05	95
23	MAT-0023	Operative encompassing access	Packaging	1540.05	115
24	MAT-0024	Assimilated optimizing utilization	Packaging	3259.10	293

Statement/Syntax

Tables and Views (1)

Procedures and Functions

SQL Functions

Fig 11: Retrieved exact data from Database

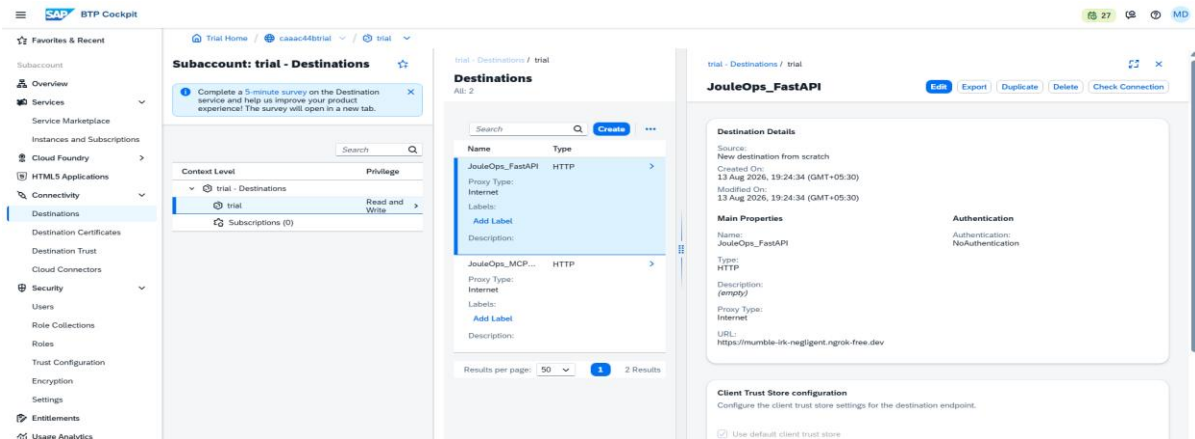


Fig 12 : SAP Destination connection creation

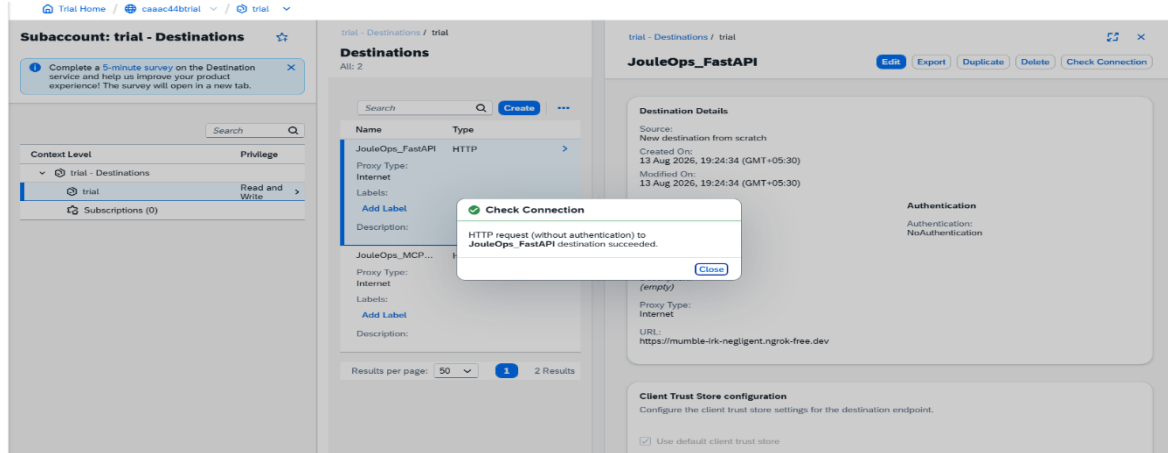


Fig 13: Connection status checking

All endpoint working outputs

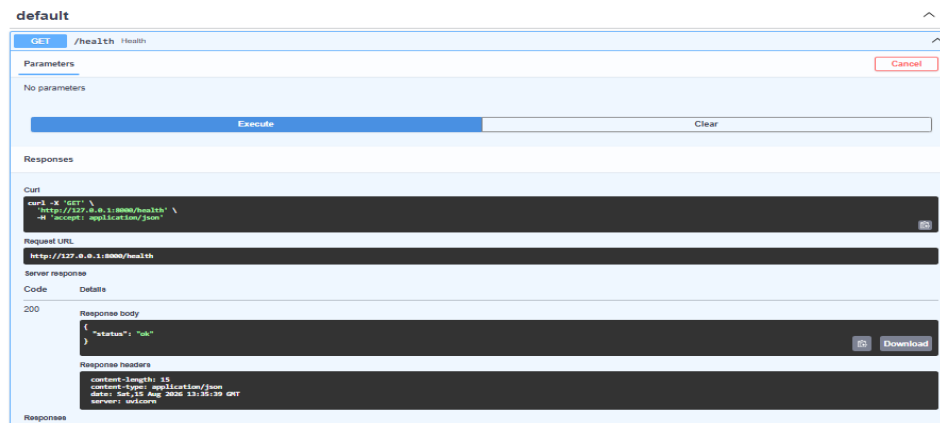


Fig 14: Health endpoint

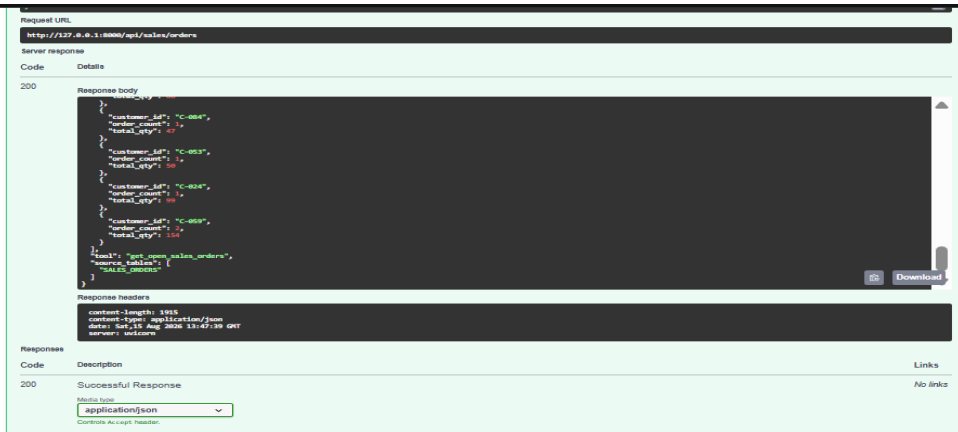


Fig 15: Sales order endpoint



Fig 16: Customer summary endpoint

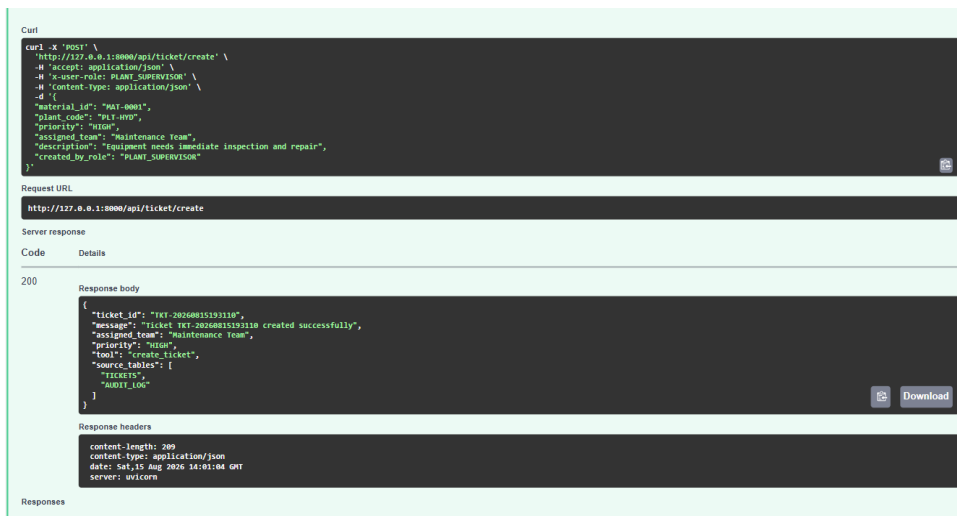


Fig 17: Ticket creation endpoint

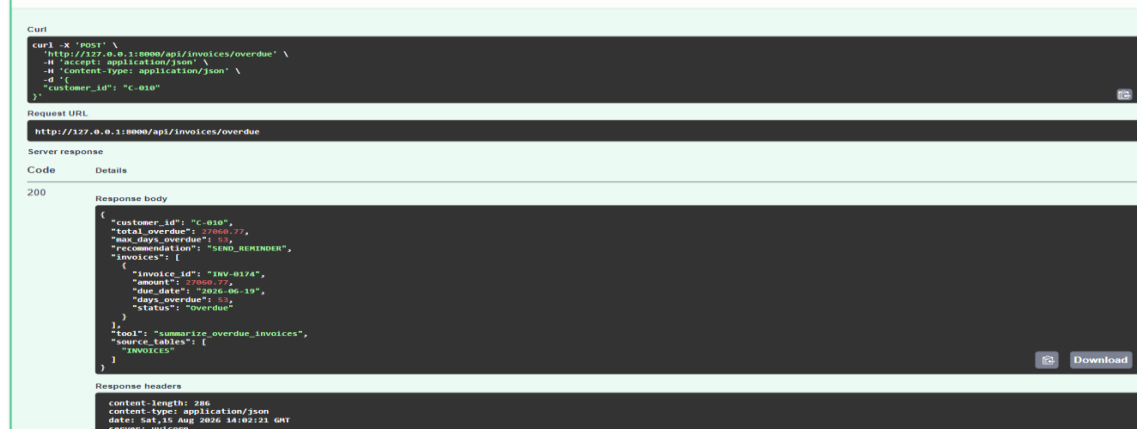


Fig 17: Invoices overdue endpoint

Joule Skills Created Using SAP Lobby from SAP learning workbook

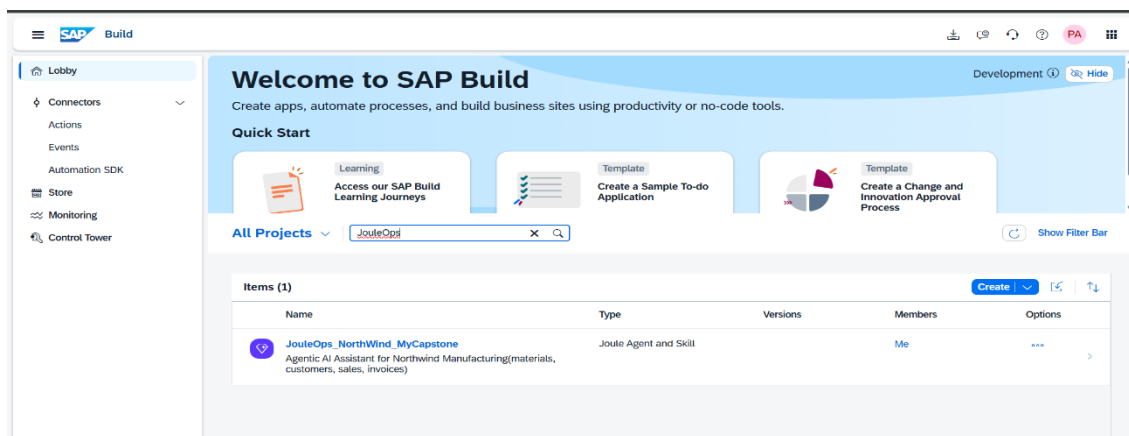


Fig 18: Project creation

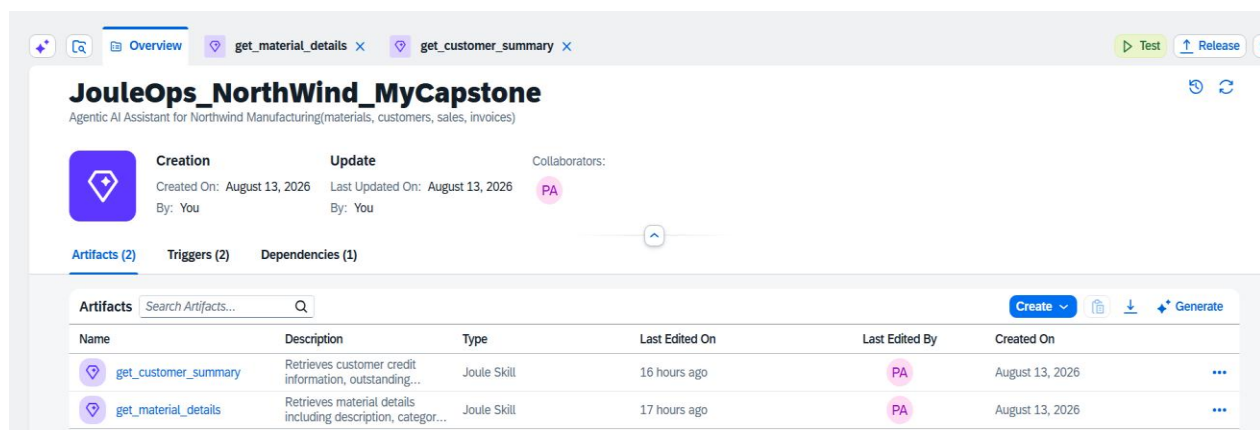


Fig 19: Sample Joule Skill creation and mapped the input and output parameters

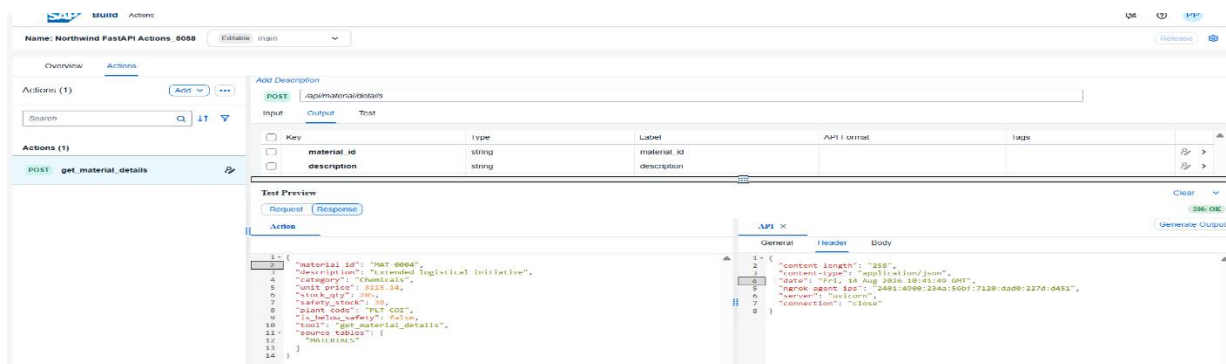


Fig 20: Output obtained for *get_material_details* joule skill